AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING

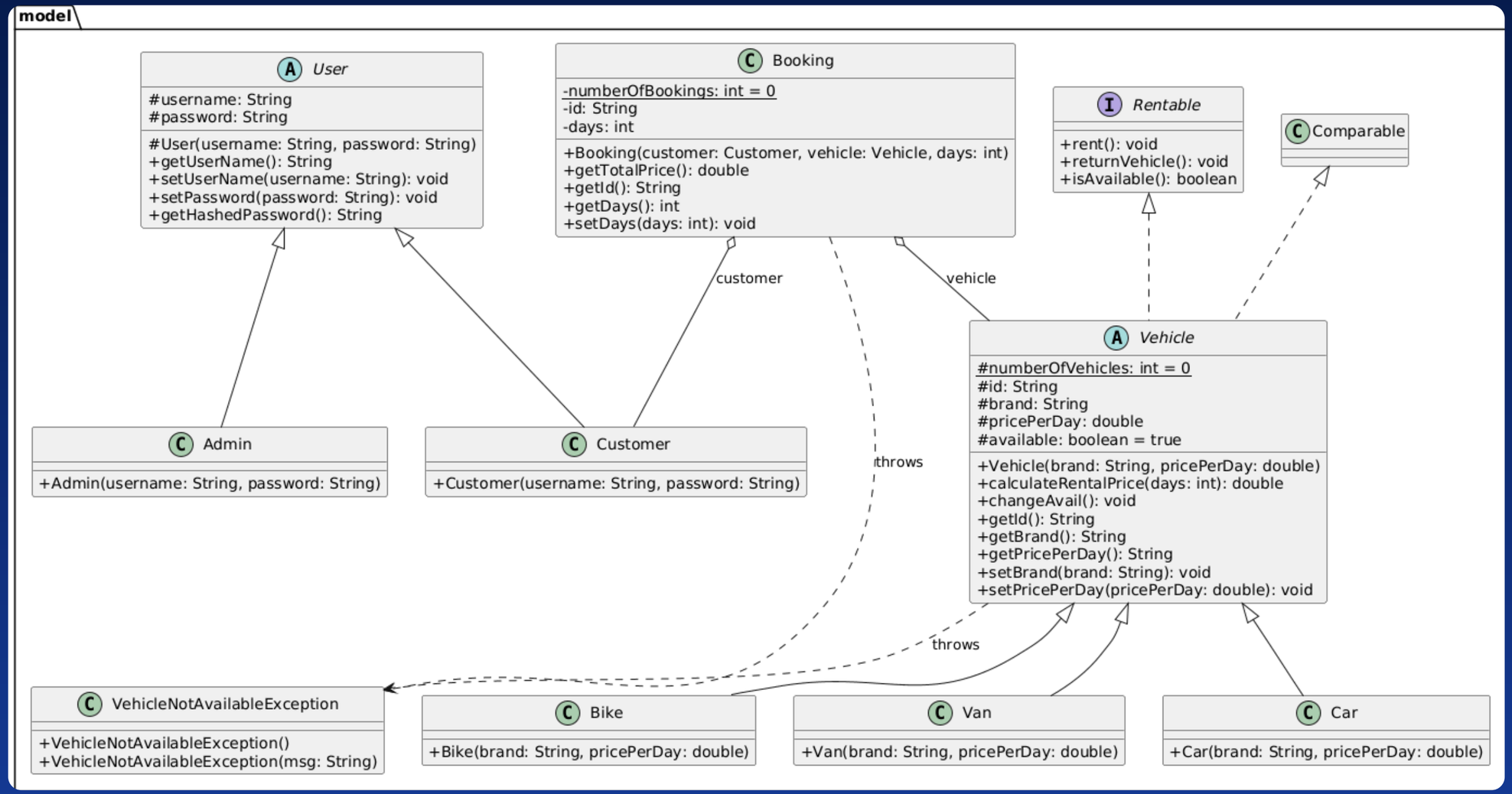
VEHCILE RENTAL SYSTEM

[CSE331S] ADVANCED COMPUTER PROGRAMMING

SYSTEM OVERVIEW:

This system is a robust, object-oriented Java application designed to manage the lifecycle of vehicle rentals, from user authentication to booking management. It follows a modular architecture that separates security concerns, data models, and business logic.

UML: CLASS DIAGRAM:



OOP CONCEPTS:

Encapsulation:

We protect sensitive data using protected fields and provide controlled access via getters and setters. Note how the password is never stored in plain text.

OOP CONCEPTS:

Encapsulation:

```java
// From User.java
protected String username;
protected String password;

public void setPassword(String password) {
    this.password = SecurityUtils.hash(password); // Hashing logic is encapsulat
}
```

OOP CONCEPTS:

Inheritance (Abstract & Concrete):

We use a clear hierarchy to promote code reuse.

- Abstract Classes: Vehicle and User (cannot be instantiated).
- Concrete Classes: Car, Bike, Van inherit from Vehicle. Admin and Customer inherit from User.

OOP CONCEPTS:

Inheritance (Abstract & Concrete):

```java
// Inheritance in model/Van.java
public class Van extends Vehicle {
    public Car(String id, String brand, double pricePerDay) {
        super(id, brand, pricePerDay); // Passes data to the abstract Vehicle
    }
}
```

OOP CONCEPTS:

Interface & Polymorphism:

The Rentable interface allows different vehicle types to be treated uniformly during the rental process.

OOP CONCEPTS:

Interface & Polymorphism:

```java
// Polymorphism in Vehicle.java
public abstract class Vehicle implements Rentable, Comparable<Vehicle> {
    @Override
    public void rent() throws VehicleNotAvailableException {
        // Implementation of interface method
    }
}
```

GENERIC SORT & COMPARABLE:

Generic Sort & Comparable:

To help users find the best deals, we implemented Comparable to sort our vehicles by price.

GENERIC SORT & COMPARABLE:

Generic Sort & Comparable:

```java
// From Vehicle.java
@Override
public int compareTo(Vehicle v) {
    // Logic: subtracts prices to return -1, 0, or 1 for sorting algorithms
    return (int)(this.pricePerDay - v.pricePerDay);
}
```

Generic Sort & Comparable:

Generic Sort & Comparable:

Demo: In the application, if we have a list of vehicles, calling Collections.sort(vehicleList) will automatically arrange them from cheapest to most expensive.

EXCEPTION HANDLING:

Exception Handling:

We ensure the system is "crash-proof" by handling logical errors, such as trying to rent a car that is already gone.

EXCEPTION HANDLING:

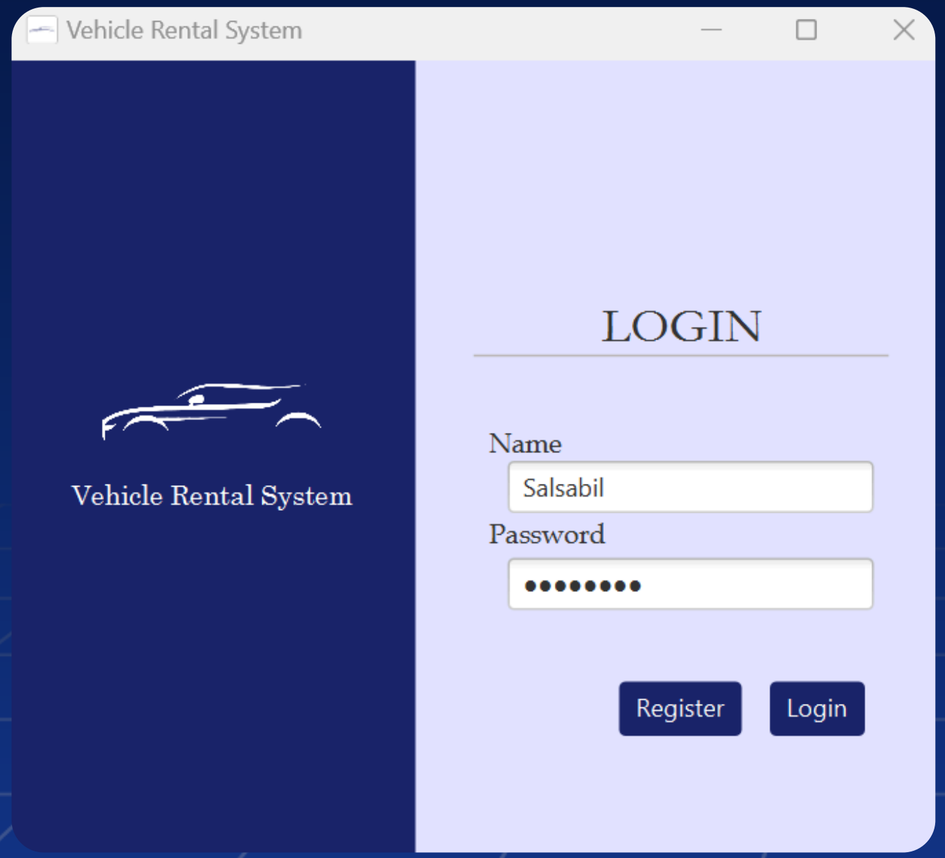
Exception Handling:

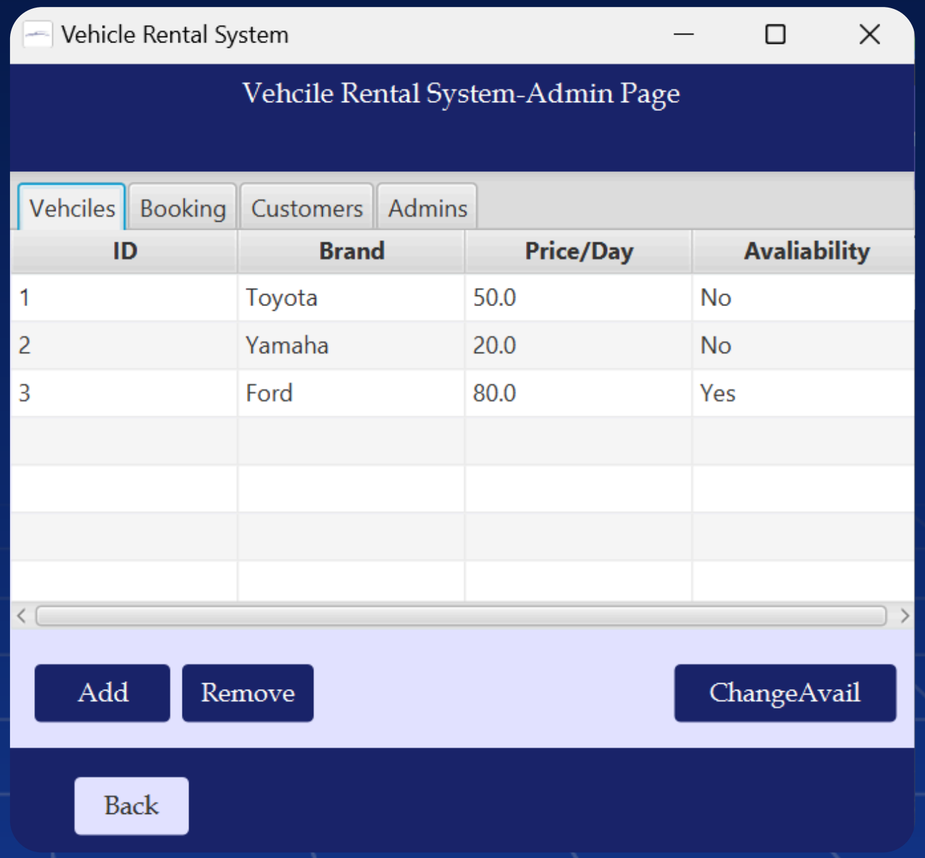
```java
// From VehicleNotAvailableException.java
public class VehicleNotAvailableException extends Exception {
    public VehicleNotAvailableException(String msg) {
        super(msg);
    }
}

// In Vehicle.java
if (!available) {
    throw new VehicleNotAvailableException("Vehicle is not available");
}
```

GUI IMPLEMENTATION

GUI Implementation:

The system starts via FinalProject.java, which uses FXMLLoader to load the MainView.fxml. We also use a SceneController to handle stage management.


Vehicle Rental System
Vehicle Rental System
LOGIN
Name
Salsabil
Password
Register
Login


Vehicle Rental System
Vehcile Rental System-Admin Page
Vehciles
Booking
Customers
Admins
ID
Brand
Price/Day
Avaliability
1
Toyota
50.0
No
2
Yamaha
20.0
No
3
Ford
80.0
Yes
Add
Remove
ChangeAvail
Back

Unit Testing:

Vehicle Test

```java
package model;

import javafx.application.Platform;
import org.junit.Test;
import org.junit.Before;
import org.junit.BeforeClass;
import static org.junit.Assert.*;
import java.util.concurrent.CountDownLatch;

public class VehicleTest {

    private Vehicle vehicle;

    // Initializes JavaFX toolkit once for the entire test class
    @BeforeClass
    public static void initJFX() throws InterruptedException {
        CountDownLatch latch = new CountDownLatch(1);
        Platform.startup(latch::countDown);
        latch.await();
    }

    @Before
    public void setUp() {
        // Instantiate using Car to test the abstract Vehicle logic
        vehicle = new Car("Toyota", 50.0);
    }

    @Test
    public void testInitialization() {
        assertEquals("Toyota", vehicle.getBrand());
        assertEquals(50.0, vehicle.getPricePerDay(), 0.01);
        assertTrue("Should be available by default", vehicle.isAvailable());
    }

    @Test
    public void testCalculateRentalPrice() {
        // 50.0 * 3 days = 150.0
        assertEquals(150.0, vehicle.calculateRentalPrice(3), 0.01);
    }
```

```java
    @Test
    public void testRentAndReturn() throws VehicleNotAvailableException {
        vehicle.rent();
        assertFalse("Vehicle should be unavailable", vehicle.isAvailable());

        vehicle.returnVehicle();
        assertTrue("Vehicle should be available again", vehicle.isAvailable());
    }

    @Test(expected = VehicleNotAvailableException.class)
    public void testDuplicateRentThrowsException() throws VehicleNotAvailableException {
        vehicle.rent();
        vehicle.rent(); // This second call must trigger the exception
    }

    @Test
    public void testCompareTo() {
        Vehicle cheap = new Car("Cheap", 20.0);
        Vehicle expensive = new Car("Expensive", 100.0);

        // compareTo returns (this.pricePerDay - v.pricePerDay)
        assertTrue(cheap.compareTo(expensive) < 0);
        assertTrue(expensive.compareTo(cheap) > 0);
    }

    @Test
    public void testChangeAvail() {
        boolean initialStatus = vehicle.isAvailable();
        vehicle.changeAvail();
        assertNotEquals(initialStatus, vehicle.isAvailable());
    }
}
```

UNIT TESTING:

Vehicle Test

```
package model;

import javafx.application.Platform;
import org.junit.Test;
import org.junit.Before;
import org.junit.Befor
import static org.juni
import java.util.concu

public class VehicleTe

    private Vehicle ve

    // Initializes Jav
    @BeforeClass
    public static voic
        CountDownLatch
        Platform.start
        latch.await();
    }

    @Before
    public void setUp
        // Instantiate
        vehicle = new
    }

    @Test
    public void testIn
        assertEquals(
        assertEquals(5
        assertTrue("Sh
    }

    @Test
    public void testCalculateRentalPrice() {
        // 50.0 * 3 days = 150.0
        assertEquals(150.0, vehicle.calculateRentalPrice(3), 0.01);
    }
```

```
    @Test
    public void testRentAndReturn() throws VehicleNotAvailableException {
        vehicle.rent();
                                                    vehicle.isAvailable());

                                                    vehicle.isAvailable());

                                                    )
                                                    ws VehicleNotAvailableException {

                                                    r the exception

                                                    ;

                                                    ePerDay)

        boolean initialStatus = vehicle.isAvailable();
        vehicle.changeAvail();
        assertNotEquals(initialStatus, vehicle.isAvailable());
    }
}
```

Tests passed: 100.00 %

6 tests passed. (0.916 s)

- model.VehicleTest passed
  - testCalculateRentalPrice passed (0.011 s)
  - testDuplicateRentThrowsException passed (0.002 s)
  - testCompareTo passed (0.001 s)
  - testRentAndReturn passed (0.0 s)
  - testInitialization passed (0.002 s)
  - testChangeAvail passed (0.001 s)

Unit Testing:

Car Test

```java
package model;

import org.junit.Test;
import static org.junit.Assert.*;

public class CarTest {

    @Test
    public void testAvailability() throws VehicleNotAvailableException {
        Car car = new Car("C1",  500);

        assertTrue(car.isAvailable());

        car.rent();
        assertFalse(car.isAvailable());

        car.returnVehicle();
        assertTrue(car.isAvailable());
    }

    @Test
    public void testRentalPrice() {
        Car car = new Car("C2",  600);
        assertEquals(1800.0, car.calculateRentalPrice(3), 0.001);
    }
}
```

Unit Testing:

Car Test

```
5   package model;
6
7   import org.junit.Test;
8   import static org.junit.Assert.*;
```

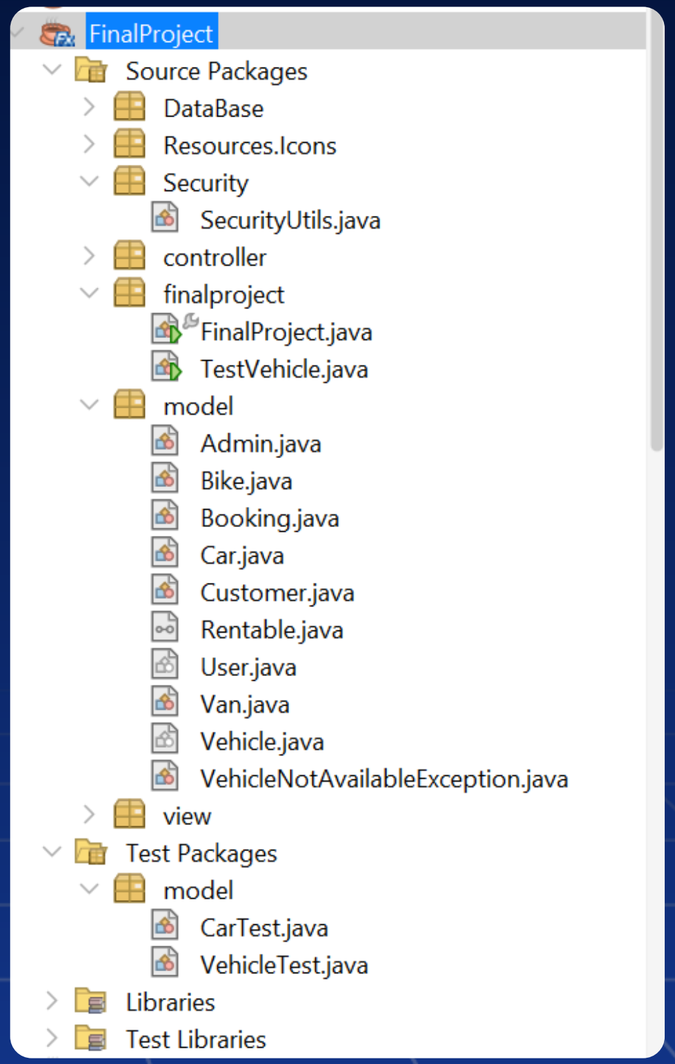
Tests passed: 100.00 %

Both tests passed. (0.302 s)

- model.CarTest passed
  - testAvailability passed (0.014 s)
  - testRentalPrice passed (0.002 s)

```
27          Car car = new Car("C2",  600);
28          assertEquals(1800.0, car.calculateRentalPrice(3), 0.001);
29      }
30  }
31
```

Project Structure



AIN SHAMS UNIVERSITY
FACULTY OF ENGINEERING

THANK YOU